

AXONA

Architecture

David A. Smith
Axona.net

An Axona Architecture Note vo.4.0 2026-05-25

A protocol is the contract between layers that don't trust each other.

What I cannot create, I do not understand.

RICHARD FEYNMAN · 1988

I The Stack

AXONA SHIPS AS THREE REPOSITORIES that compose into one running system. Each layer has a single responsibility,¹ a single canonical version identifier (`KERNEL_VERSION`), and a contract surface narrow enough to fit on a postcard.

@axona/protocol (kernel). The pure-JS library. Owns the DHT routing, the pub/sub overlay, the cryptography, and the transport contracts. No browser DOM, no Node-specific I/O, no network code outside abstract transport classes. Runs identically in browsers, in Node, in *dht-sim*'s simulated network.

axona-peer (browser application). The reference end-user app. Provides identity persistence, region selection, WebRTC mesh formation, a chat-style UI for pub/sub. Imports the kernel as a vendored copy under `vendor/axona-protocol/` so the version that ran when the page was built is the version that runs in the browser.

axona-bridge (deployed server). The signaling + introducer service at `bridge.axona.net`. Runs an embedded *AxonaPeer* of its own (so it's a full DHT participant, not just a relay), gates connecting peers behind a version handshake, relays WebRTC signaling between browsers, and forwards Axona wire frames addressed to its own embedded peer.

The three components carry their own semver tracks because each ships to a different audience: kernel changes need API-level discipline; peer changes are user-visible UI; bridge changes are operational deploys. But they all share the same kernel version, visible at runtime through `/healthz` (bridge), the `welcome` frame (any peer connecting), and the version row in the peer UI.

II Identity and Addressing

EVERY AXONA PARTICIPANT has a 264-bit identifier. The top 8 bits are an S2 geographic cell² computed from the peer's latitude and longitude; the bottom 256 bits are the SHA-256 of the peer's Ed25519 public key. A peer in London has an ID starting with `0x4f`; a peer

¹ The pattern is intentional. A peer's business logic should never need to know the wire format; a bridge should never need to know what topic a subscriber cares about; the kernel should never need to know which application is using it. Every interface in this document is a place where one layer was given the right to be wrong about the layer below.

A fourth component, *dht-sim*, runs the kernel against a simulated network for protocol research and regression benchmarks. It's not part of the production stack but shares the same *@axona/protocol* library; every change that ships to production also has to pass the simulator's 25,000-node battery.

² S2 is Google's hierarchical cell decomposition of the sphere. At level 8 the world splits into 256 cells of roughly 800 km radius each; an S2 prefix gives Axona a region-aware address without leaking GPS-level precision.

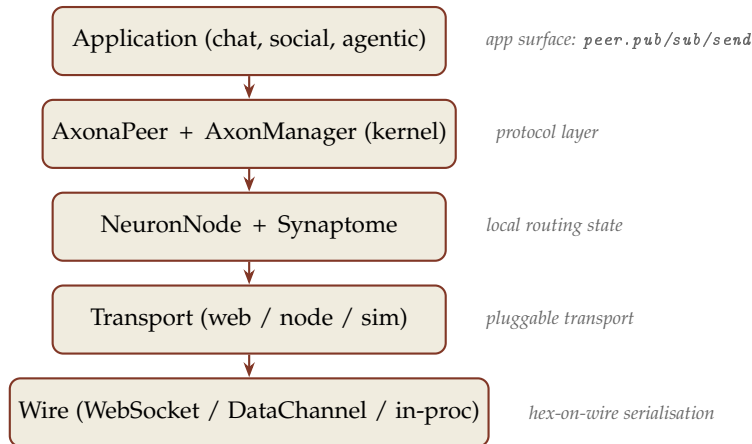


Figure 1: *

Five

layers in one peer. Every arrow is a contract; every contract has a smoke-test suite.

in Virginia starts with 0xdf. Two peers in the same region share a leading byte; two random peers share only the bits their keys happen to share.

The address space is treated as `BigInt` in every line of running code. The wire format is 66-character lowercase hex — the **ONLY** representation that ever appears in JSON. The conversion happens at exactly two places: when the transport serialises an outgoing frame, and when the dispatcher parses an incoming one.³

Topic identifiers share the address space. A topic is named by the application (a string like `us-east/hello-world`) and optionally anchored at a publisher: `topicId = sha256(publisher || topic-name)`. When the publisher is null the topic is global (top byte 00); when it's a regional synthetic publisher (top byte = S2 cell, bottom bits zero), the topic inherits the region. That's what makes regional pub/sub route naturally: peers in the same region are XOR-closest to topics anchored at that region.

The Activation Potential (AP) routing formula, the K-closest selector, and the synaptome's stratum buckets all operate on this one space. There is no separate "topic ID space" or "peer ID space"; they share addresses because they share routing.

³ This rule didn't exist in versions 1.0 through 1.4. The kernel mixed hex strings and `BigInts` in different layers and the boundaries weren't enforced; `pub/sub` silently failed because `transport.notify(bigint)` returned without sending against a hex-keyed `Map`. v1.5.0 made `BigInt`-internal canonical; `_normPeerId` helpers and `typeof v === 'bigint'` defensive branches are gone.

III The Kernel: @axona/protocol

THE KERNEL IS THE PROTOCOL. Everything else is its clients. The directory structure mirrors the contract surface:

`src/contracts/`. Abstract base classes: `Transport`, `DHT`, `DHTNode`, `BootstrapService`, `PersistenceAdapter`. These are the seams

where alternate implementations plug in. A custom transport (libp2p, QUIC, embedded) extends `Transport`; a database-backed persistence (IndexedDB, SQLite) extends `PersistenceAdapter`.

src/dht/. The active routing layer: `NeuronNode`, `Synapse`, `AxonaPeer`, `AxonaDomain`, `Subscription`. This is where the NH-1 routing logic lives, and where the developer-facing API surface (`peer.pub`, `peer.sub`, `peer.lookup`) is declared.

src/pubsub/. The pub/sub overlay: `AxonManager`, `post.js` (topic ID derivation), `envelope.js` (signed publish envelopes), `ed25519.js` (Ed25519 binding for `crypto.subtle`). `AxonManager` owns the K-closest tree, the replay caches, the dedup LRU, the metrics counters.

src/transport/. Concrete transports: `transport/web/` (browser: `BridgeTransport` over `WebSocket` + `WebRTCTransport` over `DataChannels`), `transport/node/` (server-side `WebSocket`), `transport/sim/` (in-process simulation for `dht-sim` and tests). All implement the abstract `Transport` contract identically.

src/utils/. Pure helpers: `hexid.js` (`BigInt`↔`hex` conversions, XOR distance, leading-zero count), `s2.js` (S2 cell computation), JSON codec (`transport/wire.js`).

src/identity/. Ed25519 keypair derivation + persistence envelope. The single entry point `deriveIdentity` takes a `{lat, lng}` or full options object and returns `{id, pubkey, pubkeyHex, privateKey, region, ...}`.

The whole kernel has zero runtime dependencies beyond the host JavaScript environment (browser or Node 20+). No bundler, no transpilation, no build step. The same source files run in the browser via ES modules, in Node via `direct import`, and in the simulator via the in-process transport.

IV The Local Node

EACH PEER'S VIEW OF THE NETWORK lives in two data structures on the `NeuronNode`: the *synaptome* (its outgoing connections) and the *incoming synapse index* (peers known to point at it). Both are keyed by 264-bit `BigInt` `nodeId`.

A `Synapse` is a routing edge — it carries a peer ID, a measured latency, a learned weight, a stratum (the XOR-distance bucket), an inertia lock, and provenance (was this synapse added via handshake,

hop-cache, triadic introduction, or sponsor bootstrap?). The synaptome is unbounded by design; the routing layer evicts synapses through the *Structural Survival Rule*, which guarantees navigability even under aggressive weight decay.

Routing decisions don't pick the XOR-closest synapse blindly. Each candidate is scored by Activation Potential:

$$AP_c = \frac{\Delta \text{Distance}_c}{L_c} \times (1 + w \cdot W_c)$$

The first factor is progress velocity — how much XOR distance the hop closes per millisecond of measured latency. The second is a mild boost for synapses that have historically led to fast lookups. Weight only accrues on at-or-below-average-latency paths, so a high- W synapse genuinely represents a fast route, not just a frequently-used one. Two random hops with similar latency tie; the one that's served us well before wins the tiebreak.

V The Protocol Layer

`AXONAPEER` IS THE ACTIVE OBJECT that turns a `NeuronNode` into a network participant. It owns:

- The handler installation. On `peer.start()` it registers the NH-1 routing handlers (`lookup_step`, `lookahead_probe`, `find_closest_set`, `route_msg`, `local_probe`, `triadic_introduce`, `hop_cache`, `lateral_spread`, `reinforce`) on the transport.
- The auto-admission loop. It asks `transport.boundPeers()` for currently-bound nodeIds and seeds them into the synaptome; it subscribes to `transport.onPeerBound` for future admissions.
- The pub/sub surface. `peer.pub`, `peer.sub`, `peer.pull`, `peer.metrics`: the developer-facing API. Each delegates to the `AxonManager` after deriving the topic ID and (for pub) building a signed envelope.
- The direct-message surface. `peer.send`, `peer.notify`, `peer.onMessage`: per-peer request/response and fire-and-forget, outside the pub/sub overlay. For protocols that aren't topic-shaped.
- The lookup primitive. `peer.lookup(targetKey)` runs an iterative K-closest walk and returns `{found, path, hops, peerId, latency}`. This is what `AxonManager` uses internally on every pub/sub to find the topic's axons.

`AxonManager` is a separate object owned by the peer. Its job is the pub/sub overlay: when a subscriber sends `pubsub:subscribe-k` to the K-closest axons for a topic, the receiving `AxonManager` stores the

The rule, in one sentence: a synapse may only be pruned if at least one other synapse with the same stratum exists. The effect: every populated stratum keeps at least one live route, so the address space stays globally navigable even after heavy decay.

The `AxonaPeer` surface is described in detail in the companion document *AxonaPeer Overview* (v0.4.0). This section is the architectural placement; the field-by-field description lives there.

subscriber in its `role.children` for that topic. When a publisher fans `pubsub:publish-k` to its own `K-closest`, each receiving `AxonManager` iterates its `role.children` and forwards `pubsub:deliver` to each subscriber. The publisher's signed envelope flows through unchanged; per-hop dedup prevents the fan-out tree from delivering duplicates.

The `K-closest` replication is what makes the system tolerant. A publisher sends to *five* root axons in parallel; a subscriber subscribes at *five* root axons in parallel. As long as one axon in each set overlaps, delivery succeeds. With $K = 5$ and synaptomes that converge under the bridge's peer-list dissemination, overlap is typically 4 or 5 of 5 — so a publish that loses two axons to transient WebRTC failures still delivers cleanly.

VI The Transport Layer

TRANSPORT IS WHERE ADDRESSES MEET SOCKETS. The kernel's Transport contract is a small surface: `start`, `stop`, `send` (request/response), `notify` (fire-and-forget), `isConnected`, `bindPeer`, `boundPeers`, `onPeerBound`, `onRequest`, `onNotification`, `onPeerDied`. Every concrete transport implements this surface and the upper layers don't care which one is wired in.

Three concrete implementations ship:

transport/web/. The browser stack. A `CompositeTransport` fans out to two sub-transports based on which one owns each `nodeId`: a `BridgeTransport` that carries Axona wire frames over the same `WebSocket` the bridge uses for signaling, and a `WebRTCTransport` over a `MeshManager` that owns the data-channel pool. The factory `webTransport({bridgeUrl, identity})` composes the two, drives the bridge's version gate, runs the hello/hello-ack admission, threads TURN credentials from the welcome frame into the mesh's ICE config, and resolves once everything is up.

transport/node/. A Node-side `WebSocket` transport. Used by the bridge's embedded peer to talk to connected browsers without simulating WebRTC — the bridge *is* the signaling server, so it doesn't need WebRTC to reach any browser; the `WebSocket` is direct.

transport/sim/. The simulator transport. Used by `dht-sim`'s `TransportAxonaEngine` and by the kernel's smoke tests. Identical Transport contract, but messages are scheduled in the event loop with simulated latency derived from a real internet topology dataset.

The wire format is JSON, with a single quirk: `BigInt` is serialised as "`<digits>n`" (decimal digits with an `n` suffix) and revived back to `BigInt` on decode. `NodeId` fields are always hex strings on the wire — they never appear as bigint-suffixed literals because the kernel converts them at frame construction.

The same kernel test suite runs against all three transports. Cross-transport bugs — which is to say, the kind of bug where a fix lands in one transport's call path but not the others — can't ship.

VII The Bridge

THE BRIDGE IS AN INTRODUCER, not a server. Its job is to let browsers find each other; once they've found each other, WebRTC data channels carry every byte of application traffic peer-to-peer. The bridge stays in the picture only for two reasons: (1) it's the rendezvous for the version handshake before peers can talk at all; (2) it's a guaranteed-reachable mesh peer, which makes it a useful K-closest axon in small networks where two browsers might otherwise fail to find common axons.

The bridge handles three kinds of frames over the WebSocket:

Signaling. `welcome`, `peer-list`, `peer-joined`, `peer-left`, `signal`, `ping/pong`. Routes WebRTC SDP offers/answers and ICE candidates between browsers, broadcasts join/leave events, supplies short-lived TURN credentials in the `welcome` frame.

Version handshake. `client-hello` / `server-hello`. Enforces a minimum peer version. Rejects sub-minimum peers with WebSocket close code 4426 and a human-readable reason; the peer's UI shows a "please reload" banner.

Axona wire. `{type: 'axona', payload: ...}` frames carrying `req/res/ntf` messages between any connected browser and the bridge's embedded peer. These are the same frames any two browsers exchange over WebRTC — the bridge embeds a full `AxonaPeer` of its own (`BridgeAxonaNode`) and participates in routing just like any other node, which is why it can be in K-closest sets and why it can hold subscriber children for `pub/sub` topics.

The bridge's embedded peer has a fixed identity (`cf3138...` in production, anchored at London). Its `health` surface exposes the kernel version it's running and the synaptome it has built up, available at `/healthz` JSON. Operators check that endpoint to confirm a deploy.

VIII The Wire Protocol

EVERY FRAME ON THE WIRE is JSON. Every `nodeId` or `topic ID` field is a 66-character lowercase hex string. There's no binary framing, no protobuf, no length-prefixed records — the discipline of having one wire format that any of the three transports can carry is worth the few percent of size you give up.

Three kinds of message-typed envelopes flow over the wire:

Request (*k*: `'req'`). A correlated request expecting a response.

Carries an integer `id` for correlation, a type string naming the handler (`lookup_step`, `route_msg`, `find_closest_set`, ...), and an opaque body.

Response (*k*: `'res'`). The reply to a req. Echoes the `id` and adds an `ok` boolean and a body.

Notification (*k*: `'ntf'`). Fire-and-forget. No `id`, no response. Used for `hello/hello-ack` admission, `pubsub:subscribe-k`, `pubsub:publish-k`, `pubsub:deliver`, `triadic_introduce`, `hop_cache`, `lateral_spread`, `reinforce`.

The handler types are versioned implicitly through the kernel — the bridge's `minPeerVersion` gate excludes peers running an older protocol that wouldn't understand newer messages. Major protocol changes bump `KERNEL_VERSION`'s minor; minor changes are additive (a peer that doesn't know a handler type silently drops the message).

A pub/sub message envelope is signed:

```
{
  msgId:      "<sha256 of canonical(envelope) hex>",
  ts:         1716638400000,
  topic:      "us-east/hello-world",
  message:    <application payload>,
  signerPubkey: "<Ed25519 pubkey hex>",
  signature:  "ed25519:<128 hex chars>"
}
```

Receivers verify the signature against the signer's pubkey using `crypto.subtle.verify`; the kernel surfaces verification result on the delivered envelope so applications can decide whether to trust unsigned or unverifiable publishes.

IX Routing: NH-1

NH-1 IS THE LEARNING-ADAPTIVE ROUTING LAYER. It replaces Kademlia's k -bucket array with a dynamic synaptome that adjusts edge weights based on observed latency. The basic shape — iterative lookup, $\log_2(N)$ hop count — is unchanged from Kademlia; what changes is which peer you ask at each hop.

The five wire operations that drive NH-1 are:

lookup_step(req). "Here's a target. What's your best next hop?" The receiver evaluates AP across its synaptome's progress candidates, returns the winning synapse + its own routing state.

lookahead_probe(req). "If I forward to you for target T , who would you forward to?" A one-deep peek that lets the caller compare expected hop costs across candidates before committing.

find_closest_set(req). "Give me your K peers closest to this target." The K -closest harvester used by AxonManager's pub/sub axon discovery.

route_msg(req). Generic routed-message forwarder. Used by AxonManager for `__tunneled_direct__` delivery when the target isn't directly bound.

reinforce(ntf). "This lookup succeeded; bump the weight on the synapse I used." Fire-and-forget LTP signal back along the path. Only flows when the lookup hit at-or-below average latency for the path's region.

Two structural-plasticity messages let the network rewire itself without explicit instruction: *triadic_introduce* ("introduce me to your friend at X ") and *hop_cache* ("I've forwarded to X several times; install a direct synapse to X "). Both are fire-and-forget notifications that obey the synaptome's admission rule (`_addByVitality`).

X Pub/Sub: K-Closest

THE PUB/SUB OVERLAY IS ITS OWN STRUCTURE. It sits on top of NH-1 routing but doesn't reuse the synaptome for state — each peer maintains a separate set of *axon roles*, one per topic it's chosen as an axon for. A role holds a topic ID, a set of children (subscribers), a replay cache, and a K -replicated peer set (the other root axons for the same topic).

Subscriber-side flow:

The performance claim of Axona over Kademlia is roughly 40% fewer wall-clock milliseconds per lookup at 25,000 nodes, mostly from latency-weighted hop selection. The figure is documented in the companion paper *Axona: A Learning-Adaptive DHT*.

1. Compute `topicId = sha256(publisher || topic-name)`.
2. Find the K peers closest to `topicId` in the local synaptome. (No network walk — the local reachable peer set is what matters; remote-walk K -closest returns ghost IDs that route fan-out into dead ends.)
3. Send `pubsub:subscribe-k` to each, carrying the subscriber's `nodeId` and an optional `lastSeenTs` for replay.
4. Each receiver runs `_addOrRecruitChild`, stores the subscriber under `role.children`, and optionally serves a `pubsub:replay-batch` of cached messages newer than `lastSeenTs`.

Publisher-side flow:

1. Compute the same `topicId`.
2. Build a signed envelope.
3. Find the same K peers closest to `topicId` locally.
4. Send `pubsub:publish-k` to each.
5. Each receiver runs `_onPublishDirect`: dedup against `_seenPublishes`, add to replay cache, fan `pubsub:deliver` to every child in `role.children`.
6. Each child receives `pubsub:deliver`, dedups, fires the local `_deliveryCallback` which routes to the application's subscription handler.

When a publisher's K -closest peers also happen to be in the subscribers' K -closest sets, the tree converges and delivery is single-hop from publisher to axon to subscriber. When they don't, the system relies on the redundancy: a missing axon's subtree is covered by the four other axons. The Activation Potential weighting makes high-quality axons dominate the K -closest set over time.

Per-hop `_alreadySeenPublish` dedup is keyed on `publishId` (a string like `nodeId:counter`), separate from the content-hash `msgId` the application sees. Multiple arrivals of the same `publishId` at one peer collapse to a single `_deliveryCallback` firing.

XI Identity and Security

IDENTITY IS AN ED25519 KEYPAIR. The kernel uses `crypto.subtle.generateKey` to produce a curve-Ed25519 pair; the public key is hashed with SHA-256 to produce the bottom 256 bits of the `nodeId`; the top 8 bits are the `S2` cell of the user's (`lat`, `lng`). The private key never leaves the host environment, but the public key is broadcast on every connection (in `hello/hello-ack`) so peers can verify each other.

Every signed publish carries the publisher's public key (`signerPubkey`) inline. The kernel verifies the `ed25519:<hex>` signature against `signerPubkey` using `crypto.subtle.verify`. Verification is per-envelope, not per-peer — a peer that's untrusted can still relay signed

publishes from a trusted publisher unchanged, and any receiver along the way can verify the chain of custody.

Persistence is opt-in. The kernel's `PersistenceAdapter` contract exposes `write(namespace, value)` and `read(namespace)`; concrete adapters target `IndexedDB` (browser) and the filesystem (Node). When wired, `AxonaPeer` checkpoints identity, synaptome seeds, and pending subscriptions on a 5-second debounce. On restart, the identity envelope is reloaded and validated (the stored `nodeId` must match the freshly-derived one); subscriptions are reinstalled as soon as the mesh has stabilised.

The `crypto.subtle` dependency is a hard requirement: the demo and the peer fail to boot in insecure contexts (HTTP pages outside localhost) because the Web Crypto API simply isn't available. The bridge serves over HTTPS; the peer's GitHub Pages deployment enforces HTTPS at the Pages level; the demo at `demo.axona.net` required Let's Encrypt cert provisioning before identity derivation would run at all.

Because identity is bound to a region prefix, moving geographic location changes the `nodeId`. `axona-peer`'s `identity.js` preserves the existing identity across reloads unless the user explicitly re-selects a different region. A roaming user keeps the same DHT identity for the life of the browser profile.

XII Production Deployment

THE PUBLIC STACK RUNS AT THREE URLS.

bridge.axona.net. The deployed bridge. Runs on a DigitalOcean droplet, started by `systemd`, kept current via `git pull && npm ci --omit=dev && systemctl restart`. Exposes `/healthz` for monitoring. Listens on `wss://` only; HTTP requests are redirected.

axona.net. The reference peer application, served via GitHub Pages from the `axona-peer` repo's main branch. Every push redeploys. The vendored `axona-protocol` kernel ships with the app; the version row in the UI confirms which kernel an open tab is running.

demo.axona.net. The minimal pub/sub demo, served via GitHub Pages from the `axona-protocol` repo's main branch (where `examples/minimal-pubsub-browser/` lives). Same Pages deployment pattern; HTTPS-enforced because `crypto.subtle` requires a secure context.

A coordinated release works as follows: a fix lands in `axona-protocol`, bumps `KERNEL_VERSION` and the package version, tags the commit. `axona-peer` resyncs its `vendor/axona-protocol/` from the new kernel, bumps `PEER_VERSION`, and pushes (Pages picks up). The bridge updates its `@axona/protocol` dependency in `package.json`, bumps `VERSION` in `server.js`, pushes, and the operator runs the deploy script. Every component ends up reporting the same `kernelVersion` in its visible metadata.

Every Axona component is required to surface its `KERNEL_VERSION` at a runtime-inspectable point: the bridge in `/healthz` and the welcome frame, the peer in its UI version row, the demo in its footer. The rule exists because before it did, "which kernel is everyone on?" required SSH-ing into the bridge and grepping `node_modules`.

XIII Future Directions

THE ARCHITECTURE IS STABLE. The shape of the layering, the wire format, the BigInt-canonical address discipline, the K-closest pub/sub semantics — these are not expected to change. What’s coming sits inside the existing seams:

Federated bridges. Multiple bridges in different geographic regions, gossiping peer-lists so a peer connected to one bridge can be reached from another. Single-bridge today is a deployment simplification, not a protocol constraint.

Persistent topics. Today’s replay cache survives in-memory only. A pluggable persistence layer for `axon role.replayCache` (with a TTL) lets late subscribers catch up across bridge restarts.

Topic access control. Public-mode topics (`publisher: null`) let anyone publish. Private topics (`publisher: <fixed-pubkey>`) accept publishes only from that key. A capability layer where subscribers verify per-publish signatures against an allowed-signer set lives at the application layer today; lifting it into the kernel is straightforward.

Cross-tab state sharing. Multiple tabs on the same browser today derive the same identity and form independent peers; a BroadcastChannel-based shared peer would reduce mesh load proportional to tab count.

Native transport. A C-based or Rust-based embedded kernel for IoT-class devices, sharing the wire format with the JS kernel so existing peers can talk to it without adaptation.

The job of the architecture is to make these additions land cleanly. The job of the existing layers is to keep working while they do.

Companion documents: NeuronNode Overview (v0.4.0), AxonaPeer Overview (v0.4.0), Axona Explainer (v0.4.25), and the source repositories at github.com/axona-net.